

DESAFÍO TÉCNICO DE PRÁCTICA

Construcción de soluciones sobre Sparx Enterprise Architect y Prolaborate

Pasantías Proagile 2026



Registro de Uso de IA



Ezequiel Pino

Registro de Uso de Inteligencia Artificial

Para la ejecución del desafío, utilice múltiples modelos, en distintas fases, según pruebas previas y benchmarks estudiados, para poder seleccionar el mejor modelo, para cada tarea. La idea era utilizar criterios eficientes, para poder exprimir al máximo el potencial de cada modelo. Las tareas de desarrollo tienen diferentes demandas cognitivas y estructurales, por eso, quizás implementar un modelo frontera, como Fable, nos traiga buenos resultados, pero no es efectivo económicamente, por los altos costos de consumo de créditos, o uso de API.

Para este desafío, mi objetivo fue utilizar los modelos de la forma más eficiente, cuidadosa y selectiva posible, ya que podemos obtener resultados al nivel de un modelo de frontera, pero cuidando la billetera. A continuación detallo el proceso en distintas fases

Aclaración: En la columna de “Estrategia-Prompt”, y en la columna de “Evidencia”, decidí colocar una referencia en la celda para aquellas peticiones largas. Utilizando la referencia, se puede buscar el prompt debajo del recuadro, es una decisión simplemente estética y de orden.

Fase Inicial - Documentación

Para comenzar el desafío, lo más importante era comprender en profundidad, el entorno de Enterprise Architect, y la rigurosa y detallada documentación técnica que lo acompaña. Para esto, cree un cuaderno de GeminiNotebook (Antes conocido como NotebookLM), para tener una fuente de conocimiento directa de toda la documentación técnica. Esto me ayudó a resolver dudas puntuales que surgen al momento de tomar decisiones de diseño importantes.

The screenshot displays the Sparx Enterprise Architect documentation interface. On the left, a sidebar titled 'Fuentes' (Sources) lists various documents, including 'C_Sharp_Best_Coding_Practices.pdf', 'Desafio_Tecnico_Practica_EA_Prolabor...', 'enterprise_architect_sdk.pdf', 'enterprise_architect_sdk.pdf', 'enterprise-architect-object-model.pdf', 'Guia_Creacion_Addin_Enterprise_Archit...', and 'InsideEA.pdf'. The main area is a chat window titled 'Chat' with a prompt: 'el SQL Scratch Pad en el Database Builder) 3 4 y solo permiten sentencias SELECT; cualquier intento de modificación de datos (UPDATE o DELETE) es ignorado por seguridad 5 .'. Below the prompt, the chat content is organized into sections: '1. Tablas Clave del Repositorio de EA' and '2. Cómo consultar Metadatos en EA con SQL'. The '1. Tablas Clave del Repositorio de EA' section lists several tables: 't_object', 't_connector', 't_objectproperties', and 't_package'. The '2. Cómo consultar Metadatos en EA con SQL' section includes a sub-section 'A. Consulta de Metadatos Básicos (Estado, Tipo, Estereotipo)' and a paragraph explaining that many metadata are found directly in the columns of 't_object' (Object_Type, Stereotype, Status y Alias). The right sidebar, titled 'Studio', contains various tools and options, including 'Resumen en audio', 'Resumen en video', 'Informes', 'Cuestionario', 'Tabla de datos', 'Presentación con...', 'Mapa mental', 'Tarjetas didácticas', and 'Infografía'. At the bottom right, there is a button 'Agregar nota' (Add note).

ID	Objetivo	Herramienta	Modelo	Estrategia -Prompt	Evidencia	Resultado
DOC-001	Cómo obtener solamente los elementos directos del paquete	Gemini Notebook	- Gemini 3.5 Flash	“Según el Object Model de EA, ¿qué colección contiene los elementos directamente incluidos en un Package? Confirma si Package.Elements incluye subpaquetes o si estos se gestionan separadamente.”	Evidencia: DOC-001	Mantener Class Library C#, Interop.EA, ComVisible(true), callbacks públicos y registro bajo EAAddins64 apuntando a Addino.AddinoClass.
DOC-002	Cómo validar que el usuario seleccionó realmente un paquete	Gemini Notebook	- Gemini 3.5 Flash	“Revisá en la documentación de EA cuál es la forma correcta de saber qué objeto está seleccionado en el Project Browser. Compará GetTreeSelectedPackage(), GetTreeSelectedItemType() y GetTreeSelectedObject(). Necesitamos aceptar únicamente una selección que sea realmente EA.Package.”	Evidencia: DOC-002	Se descartó usar directamente GetTreeSelectedPackage(), porque puede devolver el paquete contenedor de otro objeto seleccionado. Se decidió comprobar primero Repository.GetTreeSelectedItemType() == EA.ObjectType.otPackage y recién entonces recuperar GetTreeSelectedObject() y validarlo como EA.Package.
DOC-003	Qué hacer si Element.Update() devuelve false sin lanzar excepción	Gemini Notebook	- Gemini 3.5 Flash	“Investigá el contrato de Element.Update(). ¿Puede fallar devolviendo false sin lanzar una excepción? ¿Cómo deberíamos distinguir ese caso de una excepción?”	Evidencia: DOC-003	Se decidió evaluar explícitamente el valor booleano retornado por Update() además de usar try/catch. Un false se trata como fallo de esa fila, se informa y se continúa procesando el resto.

Evidencia

EV:DOC-001

Sparx Enterprise Architect documentation

+ Crear cuaderno

Copiar

Estadísticas

Compartir

Configuración

PRO

Fuentes

+ Agregar fuentes

Buscar fuentes nuevas en la Web

Web

Fast Research

Seleccionar todo

C_Sharp_Best_Coding_Practices.pdf

Desafio_Tecnico_Practica_EA_Prolabor...

enterprise_architect_sdk.pdf

enterprise_architect_sdk.pdf

enterprise-architect-object-model.pdf

Guia_Creacion_Addin_Enterprise_Archit...

InsideEA.pdf

Chat

Thoughts

La colección que contiene los elementos directamente incluidos en un paquete es `Package.Elements` 1 2 .

¿Se incluyen los subpaquetes en `Package.Elements` ?

No, la colección `Package.Elements` no incluye subpaquetes 2 3 . Los subpaquetes se gestionan de forma totalmente separada a través de la colección `Package.Packages` 3 .

Detalles de la Estructura en el Object Model:

1. `Package.Elements` (Colección de objetos `Element`): Contiene únicamente los elementos de modelado de UML (como clases, casos de uso, componentes, etc.) que pertenecen directamente a la raíz de ese paquete 1 - . Para crear un nuevo elemento dentro de un paquete, se utiliza el método `Package.Elements.AddNew(Name, Type)` 5 6 .

2. `Package.Packages` (Colección de objetos `Package`): Es una colección de lectura exclusiva que permite recorrer jerárquicamente todos los subpaquetes (paquetes secundarios) contenidos dentro del paquete actual 3 . Si un paquete tiene un `ParentID` de 0, significa que es un modelo raíz (sin paquete padre) 3 .

3. El Atributo `Package.Element`: Cabe destacar que, aunque un paquete se maneja mediante la clase `Package`, cada paquete tiene una propiedad llamada `Element` (de tipo `Element`) que lo conecta con su representación correspondiente en la tabla de elementos (`Package`). 3 - 7. Este objeto `Element` asociado permite consultar o modificar...

Haz una pregunta o crea algo

7 fuentes

→

Studio

Resumen en audio

Resumen en video

Informes

Cuestionario

Tabla de datos

Presentación con...

Mapa mental

Tarjetas didácticas

Infografía

Los datos de salida de Studio se guardarán aquí. Luego de agregar fuentes, haz clic para agregar resumen de audio, una guía de estudio, un mapa mental más.

Agregar nota

EV:DOC-002

Sparx Enterprise Architect documentation

+ Crear cuaderno

Copiar

Estadísticas

Compartir

Configuración

PRO

Fuentes

+ Agregar fuentes

Buscar fuentes nuevas en la Web

Web

Fast Research

Seleccionar todo

C_Sharp_Best_Coding_Practices.pdf

Desafio_Tecnico_Practica_EA_Prolabor...

enterprise_architect_sdk.pdf

enterprise_architect_sdk.pdf

enterprise-architect-object-model.pdf

Guia_Creacion_Addin_Enterprise_Archit...

InsideEA.pdf

Chat

Thoughts

una selección que sea realmente `EA.Package`

Para identificar con total certeza el objeto seleccionado en el **Project Browser** de Enterprise Architect (EA) y asegurar que sea estrictamente un `EA.Package`, la documentación oficial establece diferencias críticas entre los tres métodos consultados 1 - :

1. Comparativa de Métodos del Object Model

• `GetTreeSelectedPackage()`: No sirve para validar la selección directa. Este método retorna el paquete en el cual el objeto seleccionado en el árbol está contenido 3 4 . Por ejemplo, si el usuario tiene seleccionada una Clase (un elemento de tipo `Element`) que pertenece al Paquete "X", `GetTreeSelectedPackage()` devolverá el objeto del Paquete "X" 3 - . Utilizarlo provocaría falsos positivos al procesar elementos comunes como si fueran el paquete seleccionado.

• `GetTreeSelectedItemType()`: Es la herramienta de filtrado y validación fundamental. Este método devuelve una enumeración de tipo `ObjectType` que identifica el tipo exacto del ítem seleccionado directamente en el árbol 2 - . Sus valores de retorno posibles para la navegación en el árbol son `otDiagram`, `otElement`, `otPackage`, `otAttribute` y `otMethod` 2 - . Al validar si el tipo devuelto es estrictamente `otPackage`, garantizas que el usuario ha seleccionado físicamente un paquete en la interfaz 1 6 .

• `GetTreeSelectedObject()`: Es el método para obtener el objeto real de manera

Haz una pregunta o crea algo

7 fuentes

→

Studio

Resumen en audio

Resumen en video

Informes

Cuestionario

Tabla de datos

Presentación con...

Mapa mental

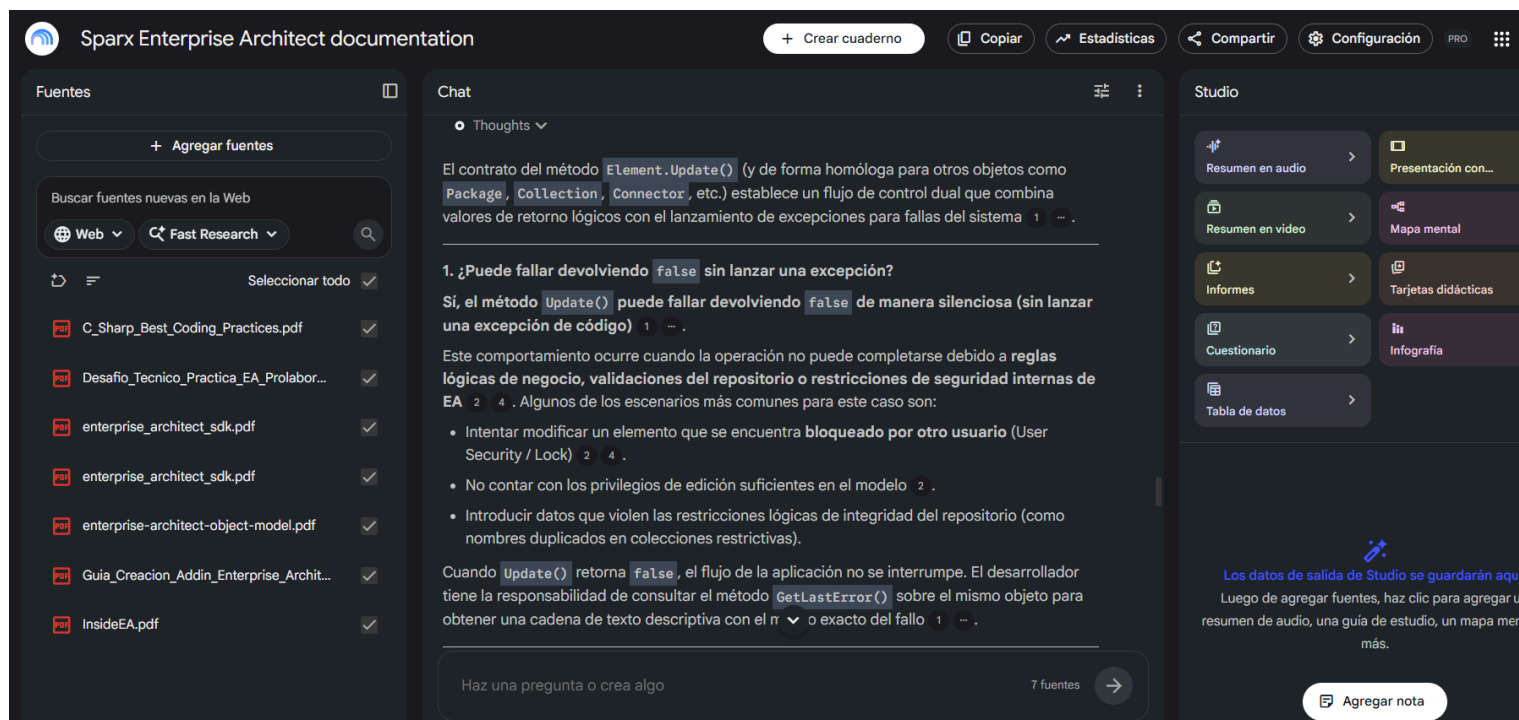
Tarjetas didácticas

Infografía

Los datos de salida de Studio se guardarán aquí. Luego de agregar fuentes, haz clic para agregar resumen de audio, una guía de estudio, un mapa mental más.

Agregar nota

EV:DOC-003



Fase de Desarrollo

Como detalle al inicio del documento, decidí utilizar modelos, de la forma más eficiente posible, con dos objetivos en mente, obtener resultados de calidad, y con el menor coste. Para esto, adoptamos la suite de Gentle-AI, que implementa la metodología de Desarrollo Guiado por Especificaciones (SDD)

En lugar de programar de forma reactiva e improvisada a partir de un chat informal, la metodología SDD estructura el ciclo de vida del desarrollo en fases lógicas y contratos formales e inmutables (Init → Proposal → Spec → Design → Tasks → Apply → Verify → Archive). Con esto, cada fase necesita los artefactos de las fases anteriores, facilitando la trazabilidad de decisiones de diseño e implementación. A continuación veamos un cuadro que resume cada fase:

Fila del TUI	UI / Abstracción anterior	Función real
gentle-orchestrator	Coordinador (no es fase)	Agente coordinador del perfil: entiende la intención, decide cuándo entrar a SDD, delega a los subagentes de cada fase y sostiene el hilo de la sesión completa.
sdd-init	Inicialización	Detecta stack, scripts, convenciones, pruebas y estado mínimo del proyecto.
sdd-explore	Exploración	Investiga flujos, dependencias, archivos clave, riesgos y áreas impactadas.

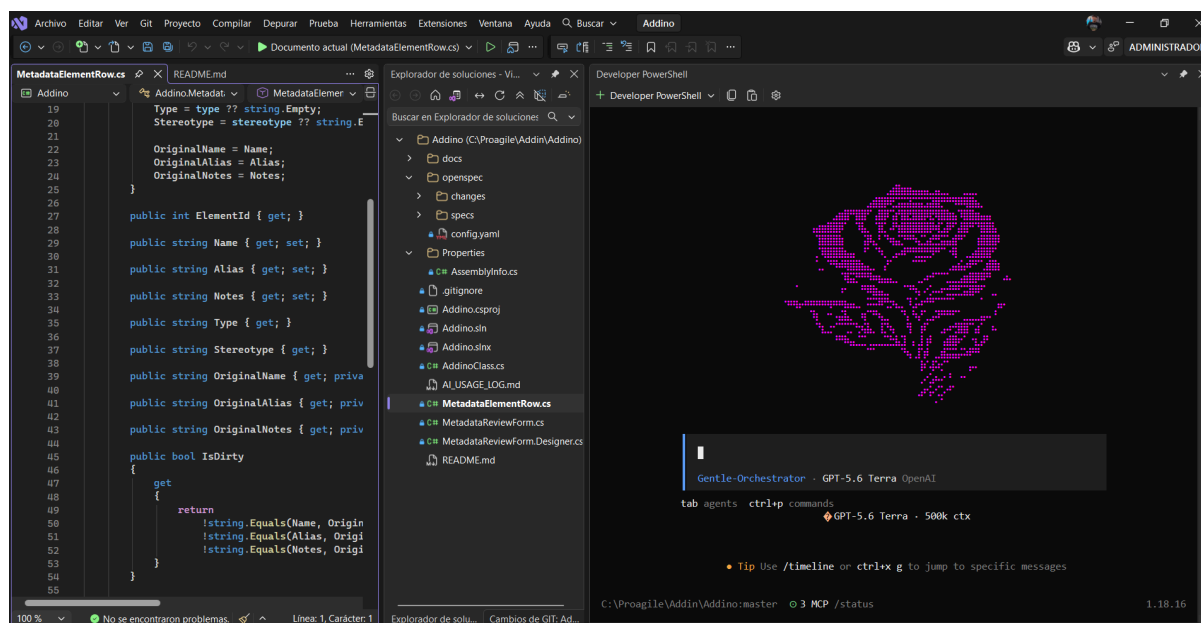
sdd-propose	Propuesta / PRD operativo	Convierte la intención en alternativa recomendada, alcance, riesgos y criterios de decisión.
sdd-spec	Especificación OpenSpec	Formaliza requisitos, contratos, criterios de aceptación y comportamiento esperado.
sdd-design	Diseño técnico	Define arquitectura, cambios por capas, estrategia de datos, APIs, UI y pruebas.
sdd-tasks	Plan de tareas	Divide el diseño en pasos ejecutables, ordenados y verificables.
sdd-apply	Implementación	Aplica cambios en código, archivos, configuraciones, tests y scripts.
sdd-verify	Review / Audit	Valida diff, pruebas, riesgos, regresiones, seguridad y cumplimiento de spec.
sdd-archive	Cierre Commit-ready	Resume artefactos, deja bitácora, notas de commit y estado final.

También usamos un sistema de memoria persistente, bautizado como ENGRAM, que resuelve la “Amnesia operativa” que pueden tener los modelos entre secciones, o al momento de recibir múltiples peticiones seguidas. ENGRAM se sitúa entre el agente, y el sistema de archivos. Si el agente intenta leer o buscar un archivo, ENGRAM intercepta la llamada y le entrega un paquete contextual estructurado y reducido en lugar de un archivo crudo completo.

A continuación dejo enlaces a repositorios de Github de este Ecosistema:

- Repositorio de Gentle-AI: <https://github.com/Gentleman-Programming/gentle-ai>
- Repositorio de ENGRAM: <https://github.com/Gentleman-Programming/engram>

De esta forma, convertimos autonomía cruda de modelos, en trabajo de ingeniería eficiente y controlado.



Utilizamos OpenCode en su forma TUI para desplegar Gentle AI en el entorno de Visual STUDIO, tal como se ve en la siguiente captura:

Se contempló utilizar la herramienta de VS Github Copilot integrada en Visual Studio, sin embargo lo descarté ya que podía controlar mejor el proceso de desarrollo utilizando Gentle-AI, utilizando SDD y control de versiones con GIT.

Selección de Modelos

Aca, asigne cada fase de SDD a distintos modelos, por tres razones fundamentales:

- Optimización de Costos de tokens
- Especialización de capacidades según tarea
- Preservación de la ventana de Contexto

Tenemos fases informativas, de síntesis, o de redacción, poner a un modelo potente, como Fable, o GPT SOL a realizar estas tareas no tiene sentido. Para esto realice una investigación donde encuentre el siguiente documento en la comunidad de Gentle-AI:

Perfiles SDD para Gentle-AI + OpenCode:

https://docs.google.com/document/d/15_b_dOgp6Wf5PI6RrZ6e0QGRBMXE2xnw/edit?usp=sharing&oid=103625650383831231991&rtpof=true&sd=true

Donde muestran una combinación de los Modelos potentes de OpenAI, con los modelos que dispone la suscripción GO de opencode. En base a este documento, y con conocimiento de pruebas anteriores que he realizado, configure mi ecosistema de la siguiente forma en la TUI de opencode:

```
Assign Models to SDD, JD & Review Agents

LM Studio discovery failed; using configured models.

Current assignments:

gentle-orchestrator (coordinator) OpenAI / GPT-5.6 Terra
Set all SDD phases (not set)
► sdd-init OpenAI / GPT-5.6 Luna
sdd-explore OpenCode Go / MiniMax-M3
sdd-propose OpenAI / GPT-5.6 Terra
sdd-spec OpenCode Go / Kimi K2.7 Code
sdd-design OpenCode Go / MiniMax-M3
sdd-tasks OpenCode Go / Qwen3.7 Plus
sdd-apply OpenCode Go / Kimi K2.7 Code
sdd-verify OpenAI / GPT-5.6 Terra
sdd-archive OpenCode Go / MiMo V2.5
sdd-onboard OpenCode Go / MiniMax-M3
--- Judgment Day ---
jd-judge-a OpenAI / GPT-5.6 Terra
jd-judge-b OpenCode Go / Qwen3.7 Max
jd-fix-agent OpenCode Go / Kimi K2.7 Code
↓ more assignments

Continue
← Back
```

De esta forma, contamos con modelos potentes, en roles como orquestador, verificador, y delegamos al resto de modelos, con criterio, el resto de fases.

Al delegar el trabajo complejo a subagentes de fase independientes (los minions), se impide que la ventana de contexto del agente orquestador se contamine con un historial masivo de exploraciones, lecturas de archivos o comandos ejecutados.

Cada subagente de fase inicia con una prompt y una ventana de contexto limpias, enfocándose estrictamente en su tarea sin arrastrar ruido, lo que asegura respuestas más estables y consistentes.

A continuación, adjunto la tabla de registro de uso que usamos para algunas fases.

Cabe aclarar que en cada fase se usó GPT 5.6 TERRA como orquestador, delegando tareas a cada modelo. Cabe recalcar que los archivos que se mencionan a continuación, se encuentran en la ruta Addino\openspec\changes\ea-metadata-review-exercise-1

ID	Objetivo	Herramienta	Modelo	Estrategia -Prompt	Evidencia	Resultado
DES-001	Inicializar formalmente el proyecto Addino dentro del flujo SDD y reconocer su estado técnico antes de planificar cambios Fase Init.	OPENCODE	GPT 5.6 LUNA	Prompts: DES-001	Estado inicial del repositorio; Addino.csproj ; AssemblyInfo.cs ; configuración SDD/OpenSpec; commit baseline funcional.	Gentle AI reconoció correctamente el proyecto existente y dejó preparado el cambio para comenzar la exploración sin modificar la funcionalidad.
DES-002	Investigar el Ejercicio 1, el baseline Addino, requisitos, riesgos y decisiones que debían resolverse antes de proponer la solución. Fase Explore	OPENCODE	Minimax - M3	Prompts: DES-002	Creación de exploration.md; desafío técnico; guía del Add-in; inspección del baseline; decisiones humanas registradas al final del artefacto.	La exploración concluyó que no existían blockers técnicos para continuar y estableció una base verificada para Proposal.

ID	Objetivo	Herramienta	Modelo	Estrategia -Prompt	Evidencia	Resultado
DES-003	Definir formalmente alcance, intención, estrategia y límites del cambio sin entrar todavía en diseño detallado. Fase Propose	OPENCODE	GPT 5.6 TERRA	Prompts: DES-003	Creación de proposal.md; exploration.md ; Evidencia: DES-003	Se aprobó un alcance concreto y proporcional, con criterios de éxito y exclusiones claramente definidos.
DES-004	Convertir Proposal y requisitos del desafío en una especificación normativa y verificable para la implementación. Fase SPEC	OPENCODE	Kimi-2, 7 Code	Prompts: DES-004	Creación de openspec/changes/ea-metadata-review-exercise-1/specs/ea-metadata-review/spec.md; Proposal; Exploration;	Se obtuvo una Spec implementable y verificable que convirtió la consigna en requisitos claros y escenarios de aceptación.
DES-005	Diseñar la arquitectura concreta de implementación respetando Proposal y Spec ya congelados.	OPENCODE	Originalmente selección Minimax - M3, pero decidí usar un modelo mas potente como GPT 5.6 SOL.	Prompts: DES-005	Creación de design.md; spec.md; proposal.md;	El diseño quedó aprobado sin preguntas abiertas y permitió pasar a Tasks con arquitectura, flujo de datos, manejo de errores y estrategia de validación definidos.

Prompts

DES-001:

Quiero comenzar el trabajo formal del desafío técnico de Proagile sobre el proyecto existente Addino. Para esta primera etapa realiza únicamente la fase de exploración SDD. No implementes código, no modifiques la funcionalidad existente y no avances a propuesta, especificación diseño ni tareas.

CONTEXTO

- Addino es un Add-in para Sparx Enterprise Architect 17.1
- Está desarrollado sobre C# sobre .NET Framework 4.7.2
- Usa Interop.EA, COM, y plataforma X64
- Existe un commit Git baseline funcional anterior a la implementación del desafío.
- La funcionalidad actual Say Hello/Say Goodbye es unicamente codigo de prueba y sera reemplazada posteriormente.

FUENTES DE VERDAD

Lee primero:

- @docs/challenge/Desafio_Tecnico_Practica_EA_Prolaborate_v2 (1).md
- @docs/challenge/Guia_Creacion_Addin_Enterprise_Architect.md

Luego inspecciona unicamente los archivos relevantes del proyecto actual. La consigna de @docs/challenge/Desafio_Tecnico_Practica_EA_Prolaborate_v2 (1).md tiene prioridad sobre CUALQUIER inferencia o recomendación.

Consulta source-material/pdf/ solo si aparece incertidumbre tecnica que necesite validación.

No hagas una revision general de todos los PDF de esta fase.

OBJETIVOS DE LA EXPLORACION

Determina

1. Estado tecnico actual de Addino y que infraestructura del Add-in de prueba puede reutilizarse.
2. Que partes de la funcionalidad actual deberan reemplazarse o ampliarse.
3. Los requisitos obligatorios, restricciones técnicas y criterios de aceptación del Ejercicio 1.
4. Los principales riesgos o incertidumbres que deban resolverse antes de rediseñar la solución, especialmente los relacionados son:
 - Selección del paquete en EA;
 - carga de elementos directos;
 - edicion en memoria;
 - cancelacion sin persistencia;
 - persistencia mediante Element.Update();
 - manejo de errores;
5. Separacion explicita entre:
 - Requisitos obligatorios;
 - Mejoras de calidad posibles;
 - Desafios opcionales;

6. Las validaciones que necesariamente deberemos realizar manualmente dentro de Enterprise Architect

RESTRICCIONES

- No implementes codigo
- No diseñes todavia una arquitectura definitiva.
- No agregues desafios opcionales al alcance base.
- No cambies Framework, plataforma, COM ni tecnologías
- No trabajes sobre el ejercicio 2 de SQL/Prolaborate, salvo reconocerlo como un entregable separado (por ahora)
- No conviertas recomendaciones o supuestos en requisitos si la consigna no los exige.
- No tomes decisiones de diseño definitivas, si detectas alternativas, déjalas como decisiones abiertas para revisión humana.

Manten la exploración proporcional al tamaño actual del proyecto. Proriza los archivos y requisitos directamente relevantes, y evita auditorías generales del entorno ya validado.

Al finalizar, persiste el artefacto de exploración, presenta un resumen conciso con hechos verificados, riesgos e incógnitas, y detente para revisión humana antes de cualquier fase posterior.

DES-002:

Realizá únicamente `sdd-explore` sobre el proyecto Addino. Leé primero el desafío técnico y la guía de creación del Add-in. Determiná el estado actual, qué puede reutilizarse, qué debe reemplazarse, requisitos obligatorios, restricciones, criterios de aceptación, riesgos sobre selección de Package, elementos directos, edición local, cancelación y `Element.Update()`, y qué debe validarse manualmente en EA. Separá obligatorios, mejoras y opcionales. No implementes código ni diseños todavía la arquitectura.”

Como complemento se incorporaron las decisiones sobre WinForms, español, `GetTreeSelectedItemType()`, `Package.Elements`, DTO local, Save explícito, errores por fila, `ShowDialog()`, `.sln` clásica y exclusión de opcionales.

DES-003

Ejecutá únicamente `sdd-propose` para `ea-metadata-review-exercise-1` usando la exploración y decisiones humanas aprobadas. Proponé el alcance mínimo necesario para cumplir el Ejercicio 1: acción de Extensions, validación estricta de `EA.Package`, grilla modal en español con elementos directos, edición mediante DTO local, persistencia explícita con `Update()`, `.sln`, README, evidencias y registro de IA. Mantené fuera del alcance los desafíos opcionales y el Ejercicio 2. No implementes código ni avances a Spec/Design

DES-004:

Ejecutá únicamente `sdd-spec` para el cambio aprobado. Convertí Proposal, Exploration y la consigna en requisitos técnicos verificables usando lenguaje MUST/SHALL y escenarios de aceptación. Especificá callbacks, validación de Package, carga de hijos directos, columnas y permisos, modalidad/idioma, ciclo local de edición, Save solo de filas modificadas, manejo de `Update() == false`/excepciones, plataforma y entregables. No diseñes clases o detalles de implementación que correspondan a Design y no incluyas opcionales.

DES-005:

Avanza Formalmente a `sdd-ddesing` para el cambio `ea-metadata-review-1`.

La Proposal y la Spec está probada y congeladas. No las modifiques ni reabras decisiones funcionales ya resueltas. El checkpoint Git previo a Design ya fue realizado.

Usa como fuentes de verdad:

- Los documentos de `@docs/challenge\`
- `@openspec/specs/ea-metadata-review/spec.md`
- `@openspec/changes/ea-metadata-review-exercise-1/proposal.md`
- `@openspec/changes/ea-metadata-review-exercise-1/exploration.md`
- El código actual del proyecto Addino.

OBJETIVO:

Diseña cómo implementar el Ejercicio 1 obligatorio sobre el Add-in existente, de forma simple, robusta, mantenible y proporcional al tamaño del desafío. El diseño debe respetar sin cambios lo siguiente:

- C# sobre .NET Framework 4.7.2.
- Windows Forms.
- EA 17.1 x64 como target validado.

- Interop.EA y registro COM.
- Los cinco callbacks EA requeridos.
- Acción en Extensions.
- validación estricta de EA.Package;
- carga exclusiva de elementos directos;
- Name/Alias editables, Notes multilínea, Type/Stereotype read-only;
- edición local hasta Save;
- Cancel/Esc/X sin persistencia;
- Element.Update() solo para elementos modificados;
- procesamiento independiente y manejo de false, excepciones y elementos bloqueados;
- UI modal y visible en español.

Decisiones que debe resolver Design

Define y justifica la estructura concreta de implementación, incluyendo:

- componentes/clases necesarias y responsabilidad de cada una;
 - flujo desde EA_MenuClick hasta la apertura del formulario;
 - forma de representar localmente una fila y conservar una identidad - estable del elemento EA sin depender de Name/Alias;
 - estrategia de BindingList/binding y detección interna de modificaciones;
 - carga de datos desde EA.Package.Elements;
 - configuración de DataGridView, incluyendo Notes multilínea;
 - cómo asegurar que una celda todavía activa quede confirmada antes de Save;
 - cómo recuperar de forma segura el EA.Element correspondiente durante la persistencia;
 - flujo detallado de Save normal, Save sin cambios y Save parcial;
- cómo limpiar el estado pendiente de los éxitos y conservar pendiente el de los fallos;
- estrategia de mensajes/resumen de guardado en español;
 - comportamiento de Cancel, Esc y X;
 - manejo de excepciones COM y liberación/uso razonable de objetos COM;
 - modificaciones previstas sobre AddinoClass.cs, nuevos archivos WinForms y Addino.csproj;

- eliminación completa de Say Hello y Say Goodbye;
- generación posterior de la solución clásica .sln;
- tratamiento de Interop.EA para no romper la configuración actualmente funcional y permitir únicamente ajustes mínimos de portabilidad si fueran necesarios.

Para cada decisión donde existan alternativas razonables, selecciona la opción más simple que satisfaga la Spec y explica brevemente por qué.

Evita sobrearquitectura: no introduzcas interfaces, repositories, dependency injection, capas genéricas, patrones o abstracciones si no aportan una necesidad concreta a este Add-in pequeño. Tampoco acoples innecesariamente la UI directamente a objetos COM si existe una alternativa local sencilla y segura.

El Repositorio Pasantías permanece externo al workspace Git y se utiliza únicamente para validación manual en Enterprise Architect.

Mantén fuera del diseño: recursividad, creación de elementos, reload, dirty-row highlighting, validación obligatoria de Name vacío y todo el Ejercicio 2/Prolaborate.

Considera los entregables únicamente en aquello que afecte al diseño o a la capacidad de ejecutar/verificar la solución; no generes ahora capturas ni video.

RESULTADO ESPERADO

Genera y persiste únicamente el artefacto correspondiente a sdd-design.

El documento debe dejar suficientemente claro:

- arquitectura/componentes;
- responsabilidades;
- flujo de datos;
- flujo de apertura;
- flujo de Save;
- manejo de errores;
- ciclo de vida del estado local;
- interacción con EA/COM;
- archivos previstos;
- decisiones técnicas y trade-offs;
- riesgos técnicos restantes y mitigaciones;
- trazabilidad con la Spec.

No implementes código, no generes tasks y no avances a `sdd-tasks` ni `sdd-apply`.

Al finalizar, resume las decisiones principales, señala únicamente blockers o riesgos reales que necesiten decisión humana y detente para revisión.

Evidencia:

DES-003

The screenshot displays a development environment with three main panels:

- Code Editor (Left):** Shows a file named `proposal.md` with the following content:


```
38 | 'Addino.csproj' | Modified | Ne
39 | 'Addino.sln' | New | Required c
40 | 'README.md' | New | Execution c
41 | 'AI Usage Log' | New | Independ
42
43 ## Risks
44
45 | Risk | Likelihood | Mitigation
46 |-----|-----|-----
47 | Absent or invalid selection | f
48 | Locked element or 'Update()' f
49 | EA environment differs | Low |
50
51 ## Rollback Plan
52
53 Use Git to revert code and config
54
55 ## Dependencies
56
57 - .NET Framework 4.7.2 and Enterp
58 - External Repositorio Pasantias
59
60 ## Success Criteria
61
62 - [ ] The add-in registers and st
63 - [ ] The Extensions-menu action
64 - [ ] Edits remain local until Se
65 - [ ] Save persists changed rows
66 - [ ] The README enables unassist
67 - [ ] An independent AI Usage Log
68
```
- File Explorer (Middle):** Shows the project structure for `Addino` (C:\Proagile\Addino\Addino). The `exploration.md` file is selected.
- Developer PowerShell (Right):** Shows the command `$ gentle-ai sdd-status ea-metadata-review-exercise-1 --cwd "C:\Proagile\Addino\Addino" --json --instructions` and its output, which includes a JSON object with schema information and a message: `Click to expand`.

The bottom status bar indicates the file is `proposal.md` and the terminal is running `Gentle-Orchestrator - GPT-5.6 Terra`.